To see that this expression is true, we observe that $\Pi_{R-S}(r)$ gives us all tuples t that satisfy the first condition of the definition of division. The expression on the right side of the set difference operator

$$\Pi_{R-S}((\Pi_{R-S}(r) \times s) - \Pi_{R-S,S}(r))$$

serves to eliminate those tuples that fail to satisfy the second condition of the definition of division. Let us see how it does so. Consider $\Pi_{R-S}(r) \times s$. This relation is on schema R, and pairs every tuple in $\Pi_{R-S}(r)$ with every tuple in s. The expression $\Pi_{R-S,S}(r)$ merely reorders the attributes of r.

Thus, $(\Pi_{R-S}(r) \times s) - \Pi_{R-S,S}(r)$ gives us those pairs of tuples from $\Pi_{R-S}(r)$ and s that do not appear in r. If a tuple t_j is in

$$\Pi_{R-S}((\Pi_{R-S}(r) \times s) - \Pi_{R-S,S}(r))$$

then there is some tuple t_s in s that does not combine with tuple t_j to form a tuple in r. Thus, t_j holds a value for attributes $R - S$ that does not appear in $r \div s$. It is these values that we eliminate from $\Pi_{R-S}(r)$.

3.2.3.4 The Assignment Operation

It is convenient at times to write a relational-algebra expression by assigning parts of it to temporary relation variables. The **assignment** operation, denoted by $\leftarrow$, works like assignment in a programming language. To illustrate this operation, consider the definition of division in Section 3.2.3.3. We could write $r \div s$ as

$$\begin{aligned} temp1 &\leftarrow \Pi_{R-S}(r) \\ temp2 &\leftarrow \Pi_{R-S}((temp1 \times s) - \Pi_{R-S,S}(r)) \\ result &= temp1 - temp2 \end{aligned}$$

The evaluation of an assignment does not result in any relation being displayed to the user. Rather, the result of the expression to the right of the $\leftarrow$ is assigned to the relation variable on the left of the $\leftarrow$. This relation variable may be used in subsequent expressions.

With the assignment operation, a query can be written as a sequential program consisting of a series of assignments followed by an expression whose value is displayed as the result of the query. For relational-algebra queries, assignment must always be made to a temporary relation variable. Assignments to permanent relations constitute a database modification. We discuss this issue in Section 3.4. Note that the assignment operation does not provide any additional power to the algebra. It is, however, a convenient way to express complex queries.

3.3 Extended Relational-Algebra Operations

The basic relational-algebra operations have been extended in several ways. A simple extension is to allow arithmetic operations as part of projection. An important extension is to allow *aggregate operations* such as computing the sum of the elements of a

customer-name	*limit*	*credit-balance*
Curry	2000	1750
Hayes	1500	1500
Jones	6000	700
Smith	2000	400

Figure 3.25 The *credit-info* relation.

set, or their average. Another important extension is the *outer-join* operation, which allows relational-algebra expressions to deal with null values, which model missing information.

3.3.1 Generalized Projection

The **generalized-projection** operation extends the projection operation by allowing arithmetic functions to be used in the projection list. The generalized projection operation has the form

$$\Pi_{F_1, F_2, \ldots, F_n}(E)$$

where E is any relational-algebra expression, and each of $F_1, F_2, \ldots, F_n$ is an arithmetic expression involving constants and attributes in the schema of E. As a special case, the arithmetic expression may be simply an attribute or a constant.

For example, suppose we have a relation *credit-info*, as in Figure 3.25, which lists the credit limit and expenses so far (the *credit-balance* on the account). If we want to find how much more each person can spend, we can write the following expression:

$$\Pi_{customer\text{-}name,\ limit\ -\ credit\text{-}balance}\ (credit\text{-}info)$$

The attribute resulting from the expression $limit - credit\text{-}balance$ does not have a name. We can apply the rename operation to the result of generalized projection in order to give it a name. As a notational convenience, renaming of attributes can be combined with generalized projection as illustrated below:

$$\Pi_{customer\text{-}name,\ (limit\ -\ credit\text{-}balance)\ \mathbf{as}\ credit\text{-}available}\ (credit\text{-}info)$$

The second attribute of this generalized projection has been given the name *credit-available*. Figure 3.26 shows the result of applying this expression to the relation in Figure 3.25.

3.3.2 Aggregate Functions

Aggregate functions take a collection of values and return a single value as a result. For example, the aggregate function **sum** takes a collection of values and returns the sum of the values. Thus, the function **sum** applied on the collection

$$\{1, 1, 3, 4, 4, 11\}$$

customer-name	*credit-available*
Curry	250
Jones	5300
Smith	1600
Hayes	0

Figure 3.26 The result of $\Pi_{customer\text{-}name,\ (limit\ -\ credit\text{-}balance)\ \mathbf{as}\ credit\text{-}available}(credit\text{-}info)$.

returns the value 24. The aggregate function **avg** returns the average of the values. When applied to the preceding collection, it returns the value 4. The aggregate function **count** returns the number of the elements in the collection, and returns 6 on the preceding collection. Other common aggregate functions include **min** and **max**, which return the minimum and maximum values in a collection; they return 1 and 11, respectively, on the preceding collection.

The collections on which aggregate functions operate can have multiple occurrences of a value; the order in which the values appear is not relevant. Such collections are called **multisets**. Sets are a special case of multisets where there is only one copy of each element.

To illustrate the concept of aggregation, we shall use the *pt-works* relation in Figure 3.27, for part-time employees. Suppose that we want to find out the total sum of salaries of all the part-time employees in the bank. The relational-algebra expression for this query is:

$$\mathcal{G}_{\mathbf{sum}(salary)}(pt\text{-}works)$$

The symbol $\mathcal{G}$ is the letter G in calligraphic font; read it as "calligraphic G." The relational-algebra operation $\mathcal{G}$ signifies that aggregation is to be applied, and its subscript specifies the aggregate operation to be applied. The result of the expression above is a relation with a single attribute, containing a single row with a numerical value corresponding to the sum of all the salaries of all employees working part-time in the bank.

employee-name	*branch-name*	*salary*
Adams	Perryridge	1500
Brown	Perryridge	1300
Gopal	Perryridge	5300
Johnson	Downtown	1500
Loreena	Downtown	1300
Peterson	Downtown	2500
Rao	Austin	1500
Sato	Austin	1600

Figure 3.27 The *pt-works* relation.

There are cases where we must eliminate multiple occurrences of a value before computing an aggregate function. If we do want to eliminate duplicates, we use the same function names as before, with the addition of the hyphenated string "**distinct**" appended to the end of the function name (for example, **count-distinct**). An example arises in the query "Find the number of branches appearing in the *pt-works* relation." In this case, a branch name counts only once, regardless of the number of employees working that branch. We write this query as follows:

$$\mathcal{G}_{\textbf{count-distinct}(branch\text{-}name)}(pt\text{-}works)$$

For the relation in Figure 3.27, the result of this query is a single row containing the value 3.

Suppose we want to find the total salary sum of all part-time employees at each branch of the bank separately, rather than the sum for the entire bank. To do so, we need to partition the relation *pt-works* into **groups** based on the branch, and to apply the aggregate function on each group.

The following expression using the aggregation operator $\mathcal{G}$ achieves the desired result:

$$_{branch\text{-}name}\mathcal{G}_{\textbf{sum}(salary)}(pt\text{-}works)$$

In the expression, the attribute *branch-name* in the left-hand subscript of $\mathcal{G}$ indicates that the input relation *pt-works* must be divided into groups based on the value of *branch-name*. Figure 3.28 shows the resulting groups. The expression $\textbf{sum}(salary)$ in the right-hand subscript of $\mathcal{G}$ indicates that for each group of tuples (that is, each branch), the aggregation function **sum** must be applied on the collection of values of the *salary* attribute. The output relation consists of tuples with the branch name, and the sum of the salaries for the branch, as shown in Figure 3.29.

The general form of the **aggregation operation** $\mathcal{G}$ is as follows:

$$_{G_1,G_2,\ldots,G_n}\mathcal{G}_{F_1(A_1),\ F_2(A_2),\ldots,\ F_m(A_m)}(E)$$

where E is any relational-algebra expression; $G_1, G_2, \ldots, G_n$ constitute a list of attributes on which to group; each F_i is an aggregate function; and each A_i is an at-

employee-name	*branch-name*	*salary*
Rao	Austin	1500
Sato	Austin	1600
Johnson	Downtown	1500
Loreena	Downtown	1300
Peterson	Downtown	2500
Adams	Perryridge	1500
Brown	Perryridge	1300
Gopal	Perryridge	5300

Figure 3.28 The *pt-works* relation after grouping.

branch-name	*sum of salary*
Austin	3100
Downtown	5300
Perryridge	8100

Figure 3.29 Result of $_{branch\text{-}name}\mathcal{G}_{\mathbf{sum}(salary)}(pt\text{-}works)$.

tribute name. The meaning of the operation is as follows. The tuples in the result of expression E are partitioned into groups in such a way that

1. All tuples in a group have the same values for $G_1, G_2, \ldots, G_n$.
2. Tuples in different groups have different values for $G_1, G_2, \ldots, G_n$.

Thus, the groups can be identified by the values of attributes $G_1, G_2, \ldots, G_n$. For each group $(g_1, g_2, \ldots, g_n)$, the result has a tuple $(g_1, g_2, \ldots, g_n, a_1, a_2, \ldots, a_m)$ where, for each i, a_i is the result of applying the aggregate function F_i on the multiset of values for attribute A_i in the group.

As a special case of the aggregate operation, the list of attributes $G_1, G_2, \ldots, G_n$ can be empty, in which case there is a single group containing all tuples in the relation. This corresponds to aggregation without grouping.

Going back to our earlier example, if we want to find the maximum salary for part-time employees at each branch, in addition to the sum of the salaries, we write the expression

$$_{branch\text{-}name}\mathcal{G}_{\mathbf{sum}(salary),\mathbf{max}(salary)}(pt\text{-}works)$$

As in generalized projection, the result of an aggregation operation does not have a name. We can apply a rename operation to the result in order to give it a name. As a notational convenience, attributes of an aggregation operation can be renamed as illustrated below:

$$_{branch\text{-}name}\mathcal{G}_{\mathbf{sum}(salary)\ \mathbf{as}\ sum\text{-}salary,\mathbf{max}(salary)\ \mathbf{as}\ max\text{-}salary}(pt\text{-}works)$$

Figure 3.30 shows the result of the expression.

branch-name	*sum-salary*	*max-salary*
Austin	3100	1600
Downtown	5300	2500
Perryridge	8100	5300

Figure 3.30 Result of $_{branch\text{-}name}\mathcal{G}_{\mathbf{sum}(salary)\ \mathbf{as}\ sum\text{-}salary,\mathbf{max}(salary)\ \mathbf{as}\ max\text{-}salary}(pt\text{-}works)$.

employee-name	*street*	*city*
Coyote	Toon	Hollywood
Rabbit	Tunnel	Carrotville
Smith	Revolver	Death Valley
Williams	Seaview	Seattle

employee-name	*branch-name*	*salary*
Coyote	Mesa	1500
Rabbit	Mesa	1300
Gates	Redmond	5300
Williams	Redmond	1500

Figure 3.31 The *employee* and *ft-works* relations.

3.3.3 Outer Join

The **outer-join** operation is an extension of the join operation to deal with missing information. Suppose that we have the relations with the following schemas, which contain data on full-time employees:

employee (*employee-name*, *street*, *city*)
ft-works (*employee-name*, *branch-name*, *salary*)

Consider the *employee* and *ft-works* relations in Figure 3.31. Suppose that we want to generate a single relation with all the information (street, city, branch name, and salary) about full-time employees. A possible approach would be to use the natural-join operation as follows:

$$employee \bowtie ft\text{-}works$$

The result of this expression appears in Figure 3.32. Notice that we have lost the street and city information about Smith, since the tuple describing Smith is absent from the *ft-works* relation; similarly, we have lost the branch name and salary information about Gates, since the tuple describing Gates is absent from the *employee* relation.

We can use the *outer-join* operation to avoid this loss of information. There are actually three forms of the operation: *left outer join*, denoted $\supset\!\bowtie$; *right outer join*, denoted $\bowtie\!\subset$; and *full outer join*, denoted $\supset\!\bowtie\!\subset$. All three forms of outer join compute the join, and add extra tuples to the result of the join. The results of the expressions

employee-name	*street*	*city*	*branch-name*	*salary*
Coyote	Toon	Hollywood	Mesa	1500
Rabbit	Tunnel	Carrotville	Mesa	1300
Williams	Seaview	Seattle	Redmond	1500

Figure 3.32 The result of $employee \bowtie ft\text{-}works$.

employee-name	*street*	*city*	*branch-name*	*salary*
Coyote	Toon	Hollywood	Mesa	1500
Rabbit	Tunnel	Carrotville	Mesa	1300
Williams	Seaview	Seattle	Redmond	1500
Smith	Revolver	Death Valley	*null*	*null*

Figure 3.33 Result of $employee \rightarrow\!\bowtie ft\text{-}works$.

$employee \rightarrow\!\bowtie ft\text{-}works$,, $employee \bowtie\!\leftarrow ft\text{-}works$, and $employee \rightarrow\!\bowtie\!\leftarrow ft\text{-}works$ appear in Figures 3.33, 3.34, and 3.35, respectively.

The **left outer join** ($\rightarrow\!\bowtie$) takes all tuples in the left relation that did not match with any tuple in the right relation, pads the tuples with null values for all other attributes from the right relation, and adds them to the result of the natural join. In Figure 3.33, tuple (Smith, Revolver, Death Valley, $null$, $null$) is such a tuple. All information from the left relation is present in the result of the left outer join.

The **right outer join** ($\bowtie\!\leftarrow$) is symmetric with the left outer join: It pads tuples from the right relation that did not match any from the left relation with nulls and adds them to the result of the natural join. In Figure 3.34, tuple (Gates, $null$, $null$, Redmond, 5300) is such a tuple. Thus, all information from the right relation is present in the result of the right outer join.

The **full outer join**($\rightarrow\!\bowtie\!\leftarrow$) does both of those operations, padding tuples from the left relation that did not match any from the right relation, as well as tuples from the right relation that did not match any from the left relation, and adding them to the result of the join. Figure 3.35 shows the result of a full outer join.

Since outer join operations may generate results containing null values, we need to specify how the different relational-algebra operations deal with null values. Section 3.3.4 deals with this issue.

It is interesting to note that the outer join operations can be expressed by the basic relational-algebra operations. For instance, the left outer join operation, $r \rightarrow\!\bowtie s$, can be written as

$$(r \bowtie s) \cup (r - \Pi_R(r \bowtie s)) \times \{(null, \ldots, null)\}$$

where the constant relation $\{(null, \ldots, null)\}$ is on the schema $S - R$.

employee-name	*street*	*city*	*branch-name*	*salary*
Coyote	Toon	Hollywood	Mesa	1500
Rabbit	Tunnel	Carrotville	Mesa	1300
Williams	Seaview	Seattle	Redmond	1500
Gates	*null*	*null*	Redmond	5300

Figure 3.34 Result of $employee \bowtie\!\leftarrow ft\text{-}works$.

employee-name	*street*	*city*	*branch-name*	*salary*
Coyote	Toon	Hollywood	Mesa	1500
Rabbit	Tunnel	Carrotville	Mesa	1300
Williams	Seaview	Seattle	Redmond	1500
Smith	Revolver	Death Valley	*null*	*null*
Gates	*null*	*null*	Redmond	5300

Figure 3.35 Result of $employee \supset\!\!\!\subset ft\text{-}works$.

3.3.4 Null Values∗∗

In this section, we define how the various relational algebra operations deal with null values and complications that arise when a null value participates in an arithmetic operation or in a comparison. As we shall see, there is often more than one possible way of dealing with null values, and as a result our definitions can sometimes be arbitrary. Operations and comparisons on null values should therefore be avoided, where possible.

Since the special value *null* indicates "value unknown or nonexistent," any arithmetic operations (such as $+, -, *, /$) involving null values must return a null result.

Similarly, any comparisons (such as $<, <=, >, >=, \neq$) involving a null value evaluate to special value **unknown**; we cannot say for sure whether the result of the comparison is true or false, so we say that the result is the new truth value *unknown*.

Comparisons involving nulls may occur inside Boolean expressions involving the and, or, and not operations. We must therefore define how the three Boolean operations deal with the truth value *unknown*.

- **and**: (*true* **and** *unknown*) = *unknown*; (*false* **and** *unknown*) = *false*; (*unknown* **and** *unknown*) = *unknown*.
- **or**: (*true* **or** *unknown*) = *true*; (*false* **or** *unknown*) = *unknown*; (*unknown* **or** *unknown*) = *unknown*.
- **not**: (**not** *unknown*) = *unknown*.

We are now in a position to outline how the different relational operations deal with null values. Our definitions follow those used in the SQL language.

- **select**: The selection operation evaluates predicate P in $\sigma_P(E)$ on each tuple t in E. If the predicate returns the value *true*, t is added to the result. Otherwise, if the predicate returns *unknown* or *false*, t is not added to the result.
- **join**: Joins can be expressed as a cross product followed by a selection. Thus, the definition of how selection handles nulls also defines how join operations handle nulls.

  In a natural join, say $r \bowtie s$, we can see from the above definition that if two tuples, $t_r \in r$ and $t_s \in s$, both have a null value in a common attribute, then the tuples do not match.

- **projection**: The projection operation treats nulls just like any other value when eliminating duplicates. Thus, if two tuples in the projection result are exactly the same, and both have nulls in the same fields, they are treated as duplicates.

  The decision is a little arbitrary since, without knowing the actual value, we do not know if the two instances of null are duplicates or not.

- **union**, **intersection**, **difference**: These operations treat nulls just as the projection operation does; they treat tuples that have the same values on all fields as duplicates even if some of the fields have null values in both tuples.

  The behavior is rather arbitrary, especially in the case of intersection and difference, since we do not know if the actual values (if any) represented by the nulls are the same.

- **generalized projection**: We outlined how nulls are handled in expressions at the beginning of Section 3.3.4. Duplicate tuples containing null values are handled as in the projection operation.

- **aggregate**: When nulls occur in grouping attributes, the aggregate operation treats them just as in projection: If two tuples are the same on all grouping attributes, the operation places them in the same group, even if some of their attribute values are null.

  When nulls occur in aggregated attributes, the operation deletes null values at the outset, before applying aggregation. If the resultant multiset is empty, the aggregate result is null.

  Note that the treatment of nulls here is different from that in ordinary arithmetic expressions; we could have defined the result of an aggregate operation as null if even one of the aggregated values is null. However, this would mean a single unknown value in a large group could make the aggregate result on the group to be null, and we would lose a lot of useful information.

- **outer join**: Outer join operations behave just like join operations, except on tuples that do not occur in the join result. Such tuples may be added to the result (depending on whether the operation is $\supset\bowtie$, $\bowtie\subset$, or $\supset\bowtie\subset$), padded with nulls.

3.4 Modification of the Database

We have limited our attention until now to the extraction of information from the database. In this section, we address how to add, remove, or change information in the database.

We express database modifications by using the assignment operation. We make assignments to actual database relations by using the same notation as that described in Section 3.2.3 for assignment.

3.4.1 Deletion

We express a delete request in much the same way as a query. However, instead of displaying tuples to the user, we remove the selected tuples from the database. We

can delete only whole tuples; we cannot delete values on only particular attributes. In relational algebra a deletion is expressed by

$$r \leftarrow r - E$$

where *r* is a relation and *E* is a relational-algebra query.

Here are several examples of relational-algebra delete requests:

- Delete all of Smith's account records.

$$depositor \leftarrow depositor - \sigma_{customer\text{-}name = \text{“Smith”}}(depositor)$$

- Delete all loans with amount in the range 0 to 50.

$$loan \leftarrow loan - \sigma_{amount \geq 0 \text{ and } amount \leq 50}(loan)$$

- Delete all accounts at branches located in Needham.

$$\begin{aligned} r_1 &\leftarrow \sigma_{branch\text{-}city = \text{“Needham”}}(account \bowtie branch) \\ r_2 &\leftarrow \Pi_{branch\text{-}name,\ account\text{-}number,\ balance}(r_1) \\ account &\leftarrow account - r_2 \end{aligned}$$

  Note that, in the final example, we simplified our expression by using assignment to temporary relations (r_1 and r_2).

3.4.2 Insertion

To insert data into a relation, we either specify a tuple to be inserted or write a query whose result is a set of tuples to be inserted. Obviously, the attribute values for inserted tuples must be members of the attribute's domain. Similarly, tuples inserted must be of the correct arity. The relational algebra expresses an insertion by

$$r \leftarrow r \cup E$$

where *r* is a relation and *E* is a relational-algebra expression. We express the insertion of a single tuple by letting *E* be a constant relation containing one tuple.

Suppose that we wish to insert the fact that Smith has $1200 in account A-973 at the Perryridge branch. We write

$$\begin{aligned} &account \leftarrow account \cup \{(\text{A-973, “Perryridge”, 1200})\} \\ &depositor \leftarrow depositor \cup \{(\text{“Smith”, A-973})\} \end{aligned}$$

More generally, we might want to insert tuples on the basis of the result of a query. Suppose that we want to provide as a gift for all loan customers of the Perryridge branch a new $200 savings account. Let the loan number serve as the account number for this savings account. We write

$$\begin{aligned} r_1 &\leftarrow (\sigma_{branch\text{-}name = \text{“Perryridge”}}(borrower \bowtie loan)) \\ r_2 &\leftarrow \Pi_{loan\text{-}number,\ branch\text{-}name}(r_1) \\ account &\leftarrow account \cup (r_2 \times \{(200)\}) \\ depositor &\leftarrow depositor \cup \Pi_{customer\text{-}name,\ loan\text{-}number}(r_1) \end{aligned}$$

Instead of specifying a tuple as we did earlier, we specify a set of tuples that is inserted into both the *account* and *depositor* relation. Each tuple in the *account* relation has an *account-number* (which is the same as the loan number), a *branch-name* (Perryridge), and the initial balance of the new account ($200). Each tuple in the *depositor* relation has as *customer-name* the name of the loan customer who is being given the new account and the same account number as the corresponding *account* tuple.

3.4.3 Updating

In certain situations, we may wish to change a value in a tuple without changing *all* values in the tuple. We can use the generalized-projection operator to do this task:

$$r \leftarrow \Pi_{F_1, F_2, \ldots, F_n}(r)$$

where each F_i is either the ith attribute of r, if the ith attribute is not updated, or, if the attribute is to be updated, F_i is an expression, involving only constants and the attributes of r, that gives the new value for the attribute.

If we want to select some tuples from r and to update only them, we can use the following expression; here, P denotes the selection condition that chooses which tuples to update:

$$r \leftarrow \Pi_{F_1, F_2, \ldots, F_n}(\sigma_P(r)) \cup (r - \sigma_P(r))$$

To illustrate the use of the update operation, suppose that interest payments are being made, and that all balances are to be increased by 5 percent. We write

$$account \leftarrow \Pi_{account\text{-}number,\ branch\text{-}name,\ balance\ *1.05}\ (account)$$

Now suppose that accounts with balances over $10,000 receive 6 percent interest, whereas all others receive 5 percent. We write

$$\begin{aligned} account \leftarrow\ & \Pi_{AN, BN,\ balance\ *1.06}\ (\sigma_{balance > 10000}\ (account)) \\ \cup\ & \Pi_{AN,\ BN\ balance\ *1.05}\ (\sigma_{balance \leq 10000}\ (account)) \end{aligned}$$

where the abbreviations AN and BN stand for *account-number* and *branch-name*, respectively.

3.5 Views

In our examples up to this point, we have operated at the logical-model level. That is, we have assumed that the relations in thecollection we are given are the actual relations stored in the database.

It is not desirable for all users to see the entire logical model. Security considerations may require that certain data be hidden from users. Consider a person who needs to know a customer's loan number and branch name, but has no need to see the loan amount. This person should see a relation described, in the relational algebra, by

$$\Pi_{customer\text{-}name,\ loan\text{-}number,\ branch\text{-}name}\ (borrower \bowtie\ loan)$$

Aside from security concerns, we may wish to create a personalized collection of relations that is better matched to a certain user's intuition than is the logical model.

An employee in the advertising department, for example, might like to see a relation consisting of the customers who have either an account or a loan at the bank, and the branches with which they do business. The relation that we would create for that employee is

$$\Pi_{branch\text{-}name,\ customer\text{-}name}\ (depositor \bowtie account) \cup \Pi_{branch\text{-}name,\ customer\text{-}name}\ (borrower \bowtie loan)$$

Any relation that is not part of the logical model, but is made visible to a user as a virtual relation, is called a **view**. It is possible to support a large number of views on top of any given set of actual relations.

3.5.1 View Definition

We define a view by using the **create view** statement. To define a view, we must give the view a name, and must state the query that computes the view. The form of the **create view** statement is

create view v **as** <query expression>

where <query expression> is any legal relational-algebra query expression. The view name is represented by v.

As an example, consider the view consisting of branches and their customers. We wish this view to be called *all-customer*. We define this view as follows:

create view *all-customer* **as**
$$\Pi_{branch\text{-}name,\ customer\text{-}name}\ (depositor \bowtie account) \cup \Pi_{branch\text{-}name,\ customer\text{-}name}\ (borrower \bowtie loan)$$

Once we have defined a view, we can use the view name to refer to the virtual relation that the view generates. Using the view *all-customer*, we can find all customers of the Perryridge branch by writing

$$\Pi_{customer\text{-}name}\ (\sigma_{branch\text{-}name = \text{“Perryridge”}}\ (all\text{-}customer))$$

Recall that we wrote the same query in Section 3.2.1 without using views.

View names may appear in any place where a relation name may appear, so long as no update operations are executed on the views. We study the issue of update operations on views in Section 3.5.2.

View definition differs from the relational-algebra assignment operation. Suppose that we define relation $r1$ as follows:

$$r1 \leftarrow \Pi_{branch\text{-}name,\ customer\text{-}name}\ (depositor \bowtie account) \cup \Pi_{branch\text{-}name,\ customer\text{-}name}(borrower \bowtie loan)$$

We evaluate the assignment operation once, and $r1$ does not change when we update the relations *depositor*, *account*, *loan*, or *borrower*. In contrast, any modification we make to these relations changes the set of tuples in the view *all-customer* as well. Intuitively, at any given time, the set of tuples in the view relation is the result of evaluation of the query expression that defines the view at that time.

Thus, if a view relation is computed and stored, it may become out of date if the relations used to define it are modified. To avoid this, views are usually implemented as follows. When we define a view, the database system stores the definition of the view itself, rather than the result of evaluation of the relational-algebra expression that defines the view. Wherever a view relation appears in a query, it is replaced by the stored query expression. Thus, whenever we evaluate the query, the view relation gets recomputed.

Certain database systems allow view relations to be stored, but they make sure that, if the actual relations used in the view definition change, the view is kept up to date. Such views are called **materialized views**. The process of keeping the view up to date is called **view maintenance**, covered in Section 14.5. Applications that use a view frequently benefit from the use of materialized views, as do applications that demand fast response to certain view-based queries. Of course, the benefits to queries from the materialization of a view must be weighed against the storage costs and the added overhead for updates.

3.5.2 Updates through Views and Null Values

Although views are a useful tool for queries, they present serious problems if we express updates, insertions, or deletions with them. The difficulty is that a modification to the database expressed in terms of a view must be translated to a modification to the actual relations in the logical model of the database.

To illustrate the problem, consider a clerk who needs to see all loan data in the *loan* relation, except *loan-amount*. Let *loan-branch* be the view given to the clerk. We define this view as

$$\textbf{create view } \textit{loan-branch} \textbf{ as } \Pi_{\textit{loan-number},\ \textit{branch-name}}\ (\textit{loan})$$

Since we allow a view name to appear wherever a relation name is allowed, the clerk can write:

$$\textit{loan-branch} \leftarrow \textit{loan-branch} \cup \{(\text{L-37, “Perryridge”})\}$$

This insertion must be represented by an insertion into the relation *loan*, since *loan* is the actual relation from which the database system constructs the view *loan-branch*. However, to insert a tuple into *loan*, we must have some value for *amount*. There are two reasonable approaches to dealing with this insertion:

- Reject the insertion, and return an error message to the user.
- Insert a tuple (L-37, “Perryridge”, *null*) into the *loan* relation.

Another problem with modification of the database through views occurs with a view such as

$$\textbf{create view } \textit{loan-info} \textbf{ as } \Pi_{\textit{customer-name},\ \textit{amount}}(\textit{borrower} \bowtie \textit{loan})$$

loan-number	*branch-name*	*amount*
L-11	Round Hill	900
L-14	Downtown	1500
L-15	Perryridge	1500
L-16	Perryridge	1300
L-17	Downtown	1000
L-23	Redwood	2000
L-93	Mianus	500
null	*null*	1900

customer-name	*loan-number*
Adams	L-16
Curry	L-93
Hayes	L-15
Jackson	L-14
Jones	L-17
Smith	L-11
Smith	L-23
Williams	L-17
Johnson	*null*

Figure 3.36 Tuples inserted into *loan* and *borrower*.

This view lists the loan amount for each loan that any customer of the bank has. Consider the following insertion through this view:

$$\textit{loan-info} \leftarrow \textit{loan-info} \cup \{(\text{“Johnson”}, 1900)\}$$

The only possible method of inserting tuples into the *borrower* and *loan* relations is to insert (“Johnson”, *null*) into *borrower* and (*null*, *null*, 1900) into *loan*. Then, we obtain the relations shown in Figure 3.36. However, this update does not have the desired effect, since the view relation *loan-info* still does *not* include the tuple (“Johnson”, 1900). Thus, there is no way to update the relations *borrower* and *loan* by using nulls to get the desired update on *loan-info*.

Because of problems such as these, modifications are generally not permitted on view relations, except in limited cases. Different database systems specify different conditions under which they permit updates on view relations; see the database system manuals for details. The general problem of database modification through views has been the subject of substantial research, and the bibliographic notes provide pointers to some of this research.

3.5.3 Views Defined by Using Other Views

In Section 3.5.1 we mentioned that view relations may appear in any place that a relation name may appear, except for restrictions on the use of views in update ex-

pressions. Thus, one view may be used in the expression defining another view. For example, we can define the view *perryridge-customer* as follows:

create view *perryridge-customer* **as**
$$\Pi_{customer\text{-}name}\ (\sigma_{branch\text{-}name = \text{“Perryridge”}}\ (all\text{-}customer))$$

where *all-customer* is itself a view relation.

View expansion is one way to define the meaning of views defined in terms of other views. The procedure assumes that view definitions are not **recursive**; that is, no view is used in its own definition, whether directly, or indirectly through other view definitions. For example, if $v1$ is used in the definition of $v2$, $v2$ is used in the definition of $v3$, and $v3$ is used in the definition of $v1$, then each of $v1$, $v2$, and $v3$ is recursive. Recursive view definitions are useful in some situations, and we revisit them in the context of the Datalog language, in Section 5.2.

Let view v_1 be defined by an expression e_1 that may itself contain uses of view relations. A view relation stands for the expression defining the view, and therefore a view relation can be replaced by the expression that defines it. If we modify an expression by replacing a view relation by the latter's definition, the resultant expression may still contain other view relations. Hence, view expansion of an expression repeats the replacement step as follows:

repeat
  Find any view relation v_i in e_1
  Replace the view relation v_i by the expression defining v_i
until no more view relations are present in e_1

As long as the view definitions are not recursive, this loop will terminate. Thus, an expression e containing view relations can be understood as the expression resulting from view expansion of e, which does not contain any view relations.

As an illustration of view expansion, consider the following expression:

$$\sigma_{customer\text{-}name=\text{“John”}}(\ perryridge\text{-}customer)$$

The view-expansion procedure initially generates

$$\sigma_{customer\text{-}name=\text{“John”}}(\Pi_{customer\text{-}name}\ (\sigma_{branch\text{-}name = \text{“Perryridge”}}\ (all\text{-}customer)))$$

It then generates

$$\sigma_{customer\text{-}name=\text{“John”}}\ (\Pi_{customer\text{-}name}\ (\sigma_{branch\text{-}name = \text{“Perryridge”}}\ (\Pi_{branch\text{-}name,\ customer\text{-}name}\ (depositor \bowtie account) \cup \Pi_{branch\text{-}name,\ customer\text{-}name}\ (borrower \bowtie loan))))$$

There are no more uses of view relations, and view expansion terminates.